Temporal Fraud Detection under Concept Drift

REACT 2026 Datathon — Method Summary — Team **Hollow Vale**

Private LB **0.55312** (10th of the private leaderboard) | Public 0.55203 | Metric: PR-AUC

1. Approach and data handling

The organisers supply only raw transaction fields, so the task is to reconstruct behaviour—what is normal for a customer, device, merchant and location—from strictly past information. We engineer 244 features over the combined stream and fit a six-member gradient-boosted tree ensemble. The decisive difficulty was not the model but the *validation*: our first submission scored 0.7252 on a conventional 62-day forward fold and 0.52708 on the leaderboard. Nothing was leaking; the fold had averaged a regime change away. Correcting that produced most of our final gain.

The data comprise 994,590 transactions (731,942 labelled, 262,648 test) at a 1.76% base rate, with test beginning the day after the labelled period ends. Features are computed over `concat(train,test)` sorted by time. The rules permit this and it is necessary: building on train alone would deprive every test row of its own recent history, making test features systematically unlike training features. Missingness (0.4–0.6% in three categoricals) is kept as an explicit level rather than imputed—it is MCAR (NA-row fraud rates 0.0199 / 0.0193 / 0.0170 against a 0.0176 base), so imputing would erase the fact that a field was absent. **No external data of any kind was used**: no outside transaction data, fraud labels, reputation lists, or pretrained weights. The one neural component, a graph embedding, is trained from scratch on competition data.

Block	Captures	n	Gain
`velocity`	entity recency, rolling counts 1h–168h	57	27.4%
`entity`	pair novelty, device sharing, diversity, movement	55	13.1%
`amount`	amount vs. the customer's own history; trailing ranks	40	48.6%
`graph`	weekly bipartite snapshot: degree, components, embeddings	29	3.4%
`behaviour`	habit/surprise, repetition, travel, account youth	28	2.4%
`temporal`	hour/day, cyclical encodings, night flag	22	2.9%
`encoding`	past-only target encoding, 7-day feedback delay	13	2.4%

Feature engineering is worth 2.4× over raw columns (0.2259 → 0.5530 on our selection window) and holds on the drifted period as much as the easy one. The sharpest signals are *relative*: an amount above 10× the customer's own past mean is 92.8% fraud (52.7× lift); a previous transaction under a minute earlier is 86.0% fraud. Merchant velocity is worthless (0.69×)—the merchants are aggregators.

Leakage safety. Every historical statistic routes through one windowing primitive, so the past-only guarantee lives in a single place. Five guards run inside the notebook: no banned columns or duplicates; no near-perfect single feature (strongest is 0.4400, where a leak sits near 1.0); a customer's first row has no history; boundary continuity; and **truncation invariance**—the pipeline is rebuilt on a truncated stream and diffed against full-stream values, so any feature reading forward in time is caught. This covers the 202 behavioural columns. The 29 graph columns are audited separately and we state this rather than omit it: 19 are exactly invariant (every structural quantity), while 10 are expressed in an SVD/embedding basis whose orientation depends on the entity index taken from the whole stream. They use no labels and earn nothing—dropping all 29 moves our selection window by +0.0020. No feature uses any fraud label, including past labels; no model, encoder or statistic is fitted on test rows.

2. Validation strategy

The failure. Fraud amounts decay across the stream while legitimate amounts do not: the fraud median falls 4,918 BDT (January) → 2,044 (May) → 944 (mid-July), while the legitimate median holds near 485 for all 37 weeks. Inside our 62-day fold, weekly AP was flat near 0.79 for six weeks then fell to 0.599 and 0.473—and those two weeks average almost exactly the leaderboard score.

The diagnosis. This is *concept drift, not covariate shift.* An adversarial train-vs-test classifier reaches AUC 0.612 on raw columns and 0.514—chance—once `account_age_days` is removed, and `amount_bdt` has PSI 0.0002 between periods. The amount *distribution* did not move; $P(\text{fraud} \mid \text{amount})$ did. That ruled out density-ratio importance weighting on principle, since it corrects a shift in $p(x)$ while assuming $p(y \mid x)$ is fixed.

The correction. Selection moved to the last 2.5 labelled weeks (67,196 rows, 1,068 positives), read at three horizons over the *same rows*: 1–17 days ahead, 46–62 (matching the far end of the real test window), and 75–91 as a stress read. Reading one horizon answers a different question than the test set asks—one configuration ranks 7th of 8 at 46–62 days and 1st at 75–91. We also quantified the window's limit: calibrated against four leaderboard results, local gains above ≈0.008 AP pass through at 29–41%, and below that the local estimate *loses its sign.* That governed every later decision.

3. Model and key technical decisions

Six members—five LightGBM configurations plus CatBoost—each fitted with three seeds (18 models). Seeds are rank-averaged within a member, then members combined by weight; ranks rather than probabilities, since average precision depends only on ordering. Round counts are pinned from the run that produced the submission rather than re-searched: early stopping sits on a plateau here, and a matrix perturbed by 3e-4 moved one member from 230 to 758 rounds for a score change of 0.0011.

- **A stale target encoder beats a fresh one.** A fraud label does not exist when a transaction occurs, but once

an investigation confirms it. A 7-day feedback delay is worth 0.008–0.028 AP on *all six* validation windows: without it, training rows read a current encoder while test rows read one frozen at the cutoff and up to 62 days stale.

- **Add ensemble members for decorrelation, not count.** We paid a submission each way: eight LightGBM variants differing only in sample weights gained +0.0023 locally and **lost 0.0025** on the board; one CatBoost gained +0.0011 locally and **gained 0.0014**. By rank correlation against the LightGBM core, CatBoost sits at 0.62 while every LightGBM variant sits at 0.74–0.91, the band the core members occupy among themselves. XGBoost sits at 0.82—a different library is not automatically a different model.
- **Blend weighting was the one lever local validation could not see.** Moving weight onto the decorrelated member ($\frac{1}{6} \to \frac{1}{2} \to 0.65 \to 0.80$) gained 0.00155, 0.00037 and 0.00016 on the board—+0.00208 total, against a same-composition noise floor measured at 0.00019. Our local window is flat across that entire range.
- **What we deliberately did not do**, each measured rather than assumed: no `scale_pos_weight` (unweighted wins 6 of 6 windows—AP reads only ordering, so upweighting positives adds no information); no SMOTE or resampling; no recency weighting or window truncation (half-lives 7–90 d, windows 45–120 d, none clearing the noise floor on the fold built to detect them); no fitted blend weights; and no entity post-processing—shrinking toward an entity's maximum measured +0.0040, but recomputing it causally, restricted to that entity's *earlier* rows, dropped it to +0.0004. The apparent gain was a July row being adjusted by a September one.

4. Results

#	Change	Public	Δ
1	201 features, hill-climb blend	0.52708	—
2	244 features, encoder delay, corrected validation	0.54151	+0.01443
3	+8 recency/window variants (13 members)	0.53901	−0.00250
4	+ disclosed integrity feature	0.54874	+0.00723
5	+ CatBoost (one decorrelated member)	0.55014	+0.00140
7	weight per family, not per member	0.55150	+0.00155
9–10	CatBoost weight 0.65 → 0.80	**0.55203**	+0.00053

Final: **private 0.55312, 10th place**, from 13th on the public board. About 58% of the gain came from rebuilding features and validation after the drift diagnosis, 24% from the disclosed feature below, 8% from blend weighting.

The private split confirmed one result independently. Our two selected files differ only in CatBoost weight (0.80 vs. 0.65) and were 0.00016 apart on the public set—below our measured noise floor, so we could not call it real and selected both endpoints rather than betting on one. The private half, a disjoint sample, put the same file ahead by 0.00040. Two independent halves agreeing means the effect was signal, not public-set luck.

5. Reproducibility and disclosure

The submitted notebook builds features, trains all 18 models and writes the submission. Its committed Kaggle run reproduces `s10_cat80.csv`—the file that scored 0.55312—to 4.4×10^{-16}, floating-point machine epsilon, with rank correlation 1.0000000000 and 100% overlap in the top 1% of the ranking. This is verified against the uploaded file, not asserted.

External models and code: none. No pretrained models or backbones, and no third-party solution code, notebooks or kernels—every line is our own. Standard open-source libraries only: pandas, NumPy, SciPy, scikit-learn, LightGBM, XGBoost, CatBoost, PyTorch, Matplotlib. PyTorch trains a small graph-embedding module *from scratch* on competition data; no weights are loaded from anywhere, and the notebook runs with internet disabled. Three references informed feature design with no code reused: the ULB fraud-detection handbook (feedback-delay risk encoding), Whitrow et al. (2009) (aggregation windows), and Bahnsen et al. (2016) (RFM and periodic time features).

`signup_inconsistency_d`**.** One feature compares a row's implied signup day against the one the customer's earlier rows established. Computed past-only from raw non-target columns it is an ordinary data-quality check, and "this account's stated age contradicts its own history" is a genuine identity-tampering signal. In this dataset it is unusually sharp: of the 1,014 labelled rows where it fires, 85.8% are fraud (48.7× lift, 6.75% of all fraud), while 0.020% of legitimate rows show any inconsistency. We measured its value directly by rebuilding the identical blend without it: **0.54548 against 0.55150, so** +0.00602. It is switchable by one environment variable and we are content to be scored without it.

We used the leaderboard to set one parameter—the blend weight above, because our local window is flat across that range. Every other decision was made locally, and most came back negative.

Limitations. Our local window cannot resolve differences below ≈0.008 AP: 1,068 positives are not enough, and a better-centred estimator predicts the board *level* eight times more accurately yet still gets a 0.0025 comparison backwards. The graph tier is also not bit-reproducible across rebuilds, which is why round counts are pinned and the exact feature matrix accompanies the notebook.